Autonomous Vehicle Competition: SLAM-Based Navigation System for Indoor Course Completion

Team Trial & Error

Isaac Edelman, Kevin Everett, Joseph Neal, Mandar Deshmukh, Sourabh Pandharpurkar, Anudeep Vedantam
Advanced Robotics (MCEN 5302)
University of Colorado Boulder
{isaac.edelman, kevin.everett, joseph.neal, MandarMahesh.Deshmukh, sourabh.pandharpurkar, anudeep.vedantam}@colorado.edu

***Abstract*— This report describes the development of an autonomous vehicle controller for an AWS DeepRacer [1] system designed to navigate a simple closed indoor course. The controller was designed to navigate our autonomous vehicle through multiple loops of this course as quickly and consistently as possible to compete in a competition setting. To meet these requirements, we implemented a hybrid navigation pipeline that combines vision-based tape following with LiDAR-based SLAM and global navigation.**

The pipeline uses an OpenCV-based [2] tape detection node to identify and follow tape markings that define the track boundaries, while a 2D LiDAR sensor enables a SLAM implementation from slam-toolbox [3] to generate an occupancy grid map of the environment. This map serves as the foundation for global path planning. Using the navigation package NAV2 [4], waypoints are set up on which the system plans global paths. These paths are sent to a controller node which moves the robot through the planned path to complete autonomous time-trial laps without human intervention.

This report outlines the key design decisions, implementation methodology, limitaions, and performance results of the autonomous controller system.

I. Introduction

The primary motivation for this project was to develop an autonomous navigation system capable of reliably completing a closed indoor course as part of a competitive robotics challenge. The competition tasked teams with achieving three consecutive laps around a predefined track while minimizing human intervention, maximizing repeatability, and maintaining consistent lap times. Performance was evaluated on speed and lap time consistency. Additionally, in the competition format, penalties were imposed for collisions or the need for human intervention to correct the robot's trajectory. These requirements led our team to develop a framework that could balance vehicle speed with accurate localization and navigation. Our team, *Trial and Error*, therefore approached this challenge by developing a control system centered on SLAM and global path planning.

Given the competition requirements, the initial objective of our team was to use `ROS2` [5] based SLAM and navigation packages to autonomously complete the timed laps, with secondary goals involving camera-based navigation and LiDAR-driven obstacle avoidance. As the project progressed, the system evolved naturally into a dual-mode navigation architecture that combined robust, vision-based tape following with LiDAR-driven mapping and planning. The final controller is built using `ROS2` [5] tools including `slam_toolbox` [3] and the `NAV2` [4] navigation framework, while also incorporating visual tape detection and following through an Intel RealSense [6] camera and a controller node which publishes angular and horizontal velocity commands to move our autonomous vehicle along a planned path. This approach allowed high-level map-based localization and planning and enabled the robot to navigate through a simple course without human intervention.



Fig. 1. AWS DeepRacer [1] with RP Lidar A1, Real Sense D435i camera, Raspberry Pi 4b [18], and Pixhawk [10] flight controller.

II. System Architecture

The autonomous navigation system is built upon the AWS DeepRacer [1] platform, utilizing a distributed software architecture managed by ROS2 Humble. The autonomous vehicle hardware used is outlined below:

Compute Architecture

- Raspberry Pi 4 Model B [18]
  - 4 GB RAM
  - OS: Ubuntu 22.04 [17]
  - ROS Distribution: ROS2 Humble

Sensor Suite

- RP Lidar A1 (360° 2D LiDAR)
  - Range: 0.15 m – 12 m
  - Scan rate: 5.5 Hz
  - Angular resolution: $\sim$1°
  - Connection: USB serial (`/dev/ttyUSB0`)
- Intel RealSense D435i (RGB-D Camera) [6]
  - Resolution: 640×480 @ 30 fps
  - Field of View: 87° × 58°
  - Connection: USB 3.0

Control

- Pixhawk [10] 4 Mini Flight Controller
  - Firmware: ArduPilot Rover 4.x
  - IMU: ICM-20689 (accelerometer + gyroscope)
  - Connection: USB serial (`/dev/ttyACM1`, 115200 baud)
  - Control protocol: MAVLink [11]

This hardware allows our system to perform SLAM, global navigation, and reactive control. The Rasberry Pi onboard computer manages the full `ROS2` software stack [5] and is responsible for orchestrating perception, localization, planning, and control processes. The RealSense [6] camera incorporates both range-based and vision-based sensing to provide complementary information for mapping, obstacle awareness, and tape detection, however, we only utilized the RGB functionality of the camera. Vehicle actuation is handled by the Pixhawk [10] low-level controller running embedded firmware and communicating with `ROS2` [5] through a standardized messaging interface for issuing velocity and steering commands. Within this architecture, high-level modules including SLAM, global path planning, and motor control operate based upon the fused sensor data, while the low-level controller ensures stable execution of motion commands. This division of responsibilities provides the computational resources, sensing, and control responsiveness required for reliable autonomous navigation in the indoor environment in which our autonomous vehicle operates.

A. ROS 2 Communication Architecture

Figure 2 illustrates the complete data flow through our autonomous navigation system, organized into three functional layers: sensing, processing, and actuation. At the sensing layer, three hardware components provide complementary perception capabilities: the Pixhawk [10] IMU for motion estimation, the RPLidar [9] for environmental mapping, and the RealSense [6] camera for visual tape detection. These sensors connect to dedicated driver nodes (`rover_node`, `rplidar_node`) [9] that abstract hardware-specific protocols and publish standardized `ROS2` [5] messages.

The processing layer implements our dual-mode navigation strategy through parallel pipelines. the vision

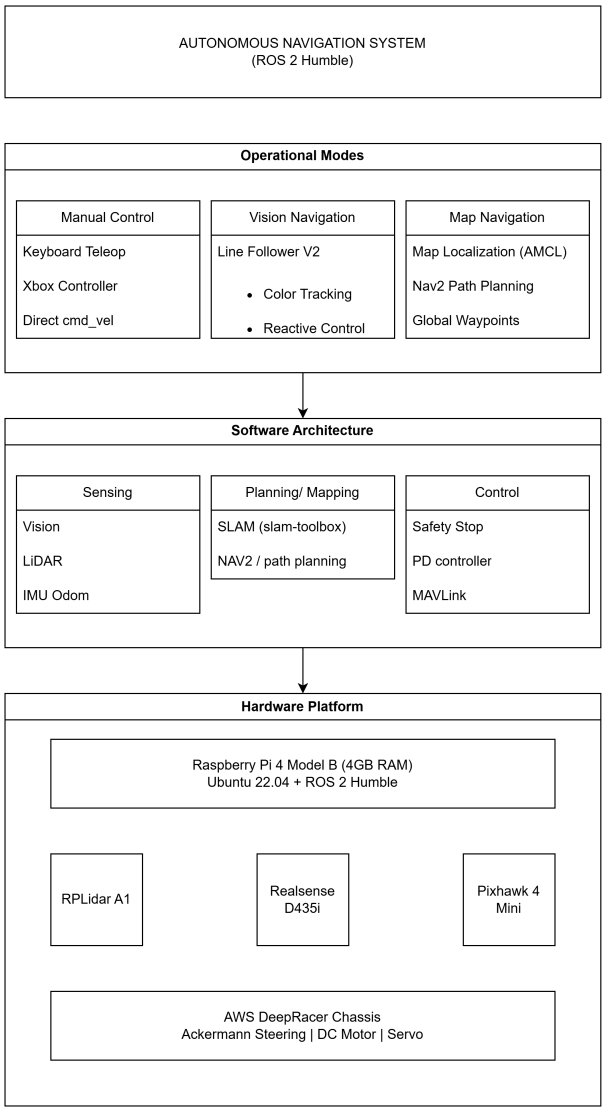


Fig. 2. High-level ROS2 [5]communication architecture organized into three functional layers: Perception (sensor data processing), Mapping/Planning (SLAM and navigation), and Control (safety and hardware interface).

pipeline (`line_follower_v2`) provides reactive tape-following behavior as an independent navigation mode. While the robot follows the tape lines, the second SLAM pipeline (`rf2_to_odometry` → `slam_toolbox`) [5] [3] builds a 2d-occupancy grid, where scan-matching odometry simultaneously updates the robot position and builds a map of the course. After a map is saved, the vehicle is switched into ”navigation mode”. In this mode, the vehicle utilizes a `NAV2` [4] package in conjunction with the saved map data to execute path planning. The path is divided into numerous ”waypoints” which are sent to a custom controller that publishes a `/cmd_vel` topic, where velocity commands move the robot through the course as quickly and consistently as possible. This architecture allows us to seperate our robots goals into two different modes, one being a mapping phase where our initial map is built, and

the second being the navigation phase in which the robot plans a path and moves through the course to achieve its goal positions.

The following diagram shows each node involved in the controller and how they communicate.

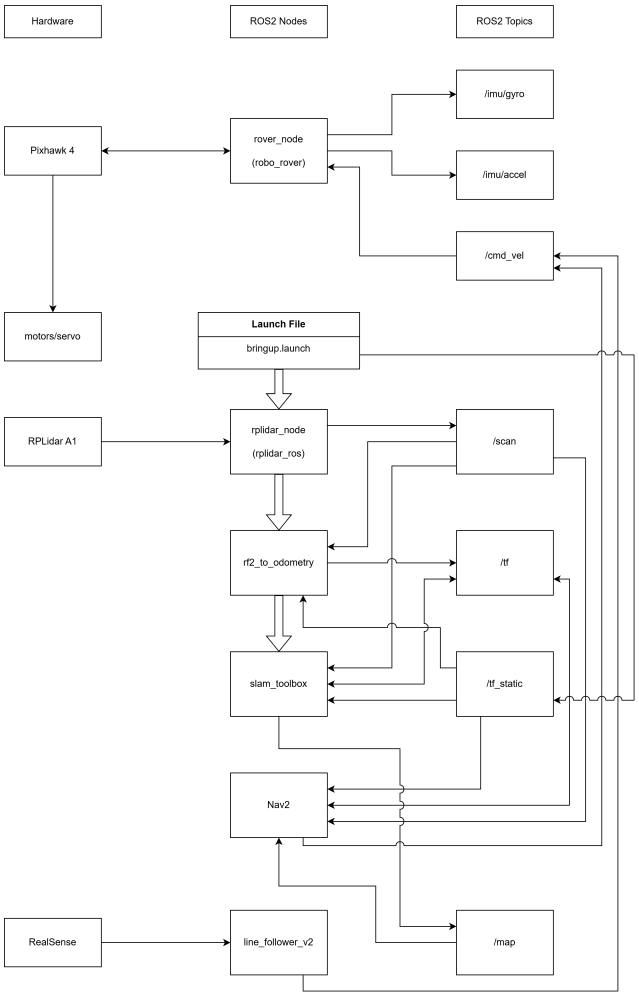


Fig. 3. Diagram of ROS2 nodes [5]

Using this infrastructure, our software framework consists of three major components:

B. Blue Tape Following Node

A custom ROS2 [5] node built with OpenCV [2] processes images from the vehicle's front-facing camera. The system:

- Converts frames to HSV color space.
- Thresholds using color ranges tuned for the blue tape.
- Extracts the tape contour and computes centroid location and line of best fit.
- Outputs a steering command proportional to the tape offset.

This module enables reactive line-following behavior and serves as a baseline navigation mode.

C. LiDAR Mapping and SLAM

A 2D occupancy grid map was created using the robot's onboard LiDAR. We used `slam_toolbox` [3] to:

- Perform online SLAM during mapping runs.
- Generate a serialized map for later use.
- Produce stable localization estimates via pose-graph optimization.

The SLAM system operates in two distinct modes. During *mapping mode*, the robot is controlled by the line follower node, navigating around the course while `slam_toolbox` incrementally builds an occupancy grid through scan matching and pose graph optimization. Raw laser scans from the RPLidar (5.5 Hz) are aligned with the existing map using correlative scan matching followed by ICP refinement. Each successful alignment creates a node in the pose graph connected by odometry-derived edges. When the robot revisits previously mapped areas, loop closure detection identifies geometric similarities and adds constraints between distant nodes, triggering graph optimization to distribute accumulated drift globally. This ensures that the final map maintains geometric consistency throughout the course, producing a 5cm resolution occupancy grid saved as `map.pgm`, `map.yaml`, and `pose_graph.data` files.

For autonomous navigation, SLAM switches to *localization mode*, loading the frozen map without further updates to the occupancy grid. Instead of building new map data, incoming laser scans are matched against the static map to estimate the robot's global pose. This scan-to-map correlation provides the `map` $\rightarrow$ `odom` transform at 10 Hz, anchoring the robot's local odometry frame to the global map coordinate system. Combined with IMU-based odometry providing the `odom` $\rightarrow$ `base_link` transform, this completes the TF tree required by Nav2 for coordinate-aware planning and control. Localization mode proved more challenging than mapping, with pose estimation reliability varying based on environment similarity and initial pose accuracy provided through RViz2 [19].

D. Global Navigation with NAV2

Once the map was generated, navigation shifted to a global-planning mode. `NAV2` [4] provided:

- Global path planning
- Local trajectory generation
- Recovery behaviors and collision avoidance

The navigation pipeline begins with the AMCL algorithm, which maintains the robot's pose estimate within the static map frame. AMCL operates by distributing particles across the map space and weighting them based on how well predicted laser scans match actual sensor readings. An initial pose estimate is provided manually through RViz2 [19], after which AMCL continuously refines the robot's position as it moves through the environment. This probabilistic approach enables robust localization even when odometry accumulates drift.

For path planning, we configured Nav2's [4] global planner to generate collision-free trajectories on the saved

occupancy grid between manually specified waypoints in RViz2 [19]. The planner accounts for the robot's footprint and applies inflation layers around obstacles to maintain safe clearance. A two-layer planning approach combines deliberate global route optimization with reactive local adjustments: the global planner computes optimal paths while the local planner monitors laser scan data, updating a local costmap to handle dynamic obstacles such as people crossing the track. Navigation parameters including maximum velocities, acceleration limits, and costmap configurations were tuned iteratively to balance speed with localization stability. The system publishes velocity commands at 10 Hz to the `/cmd_vel` topic for hardware actuation, enabling autonomous completion of multiple track laps using only the pre-built map and real-time sensor feedback.

These components enable the robot to autonomously follow predetermined waypoints for time-trial laps. The implementation of each component is described in the following section.

III. Implementation

A. Line-Following Algorithm

The line-following system processes color images from an Intel RealSense [6] camera and transforms them into a bird's-eye-view (BEV) representation of the floor. Each frame is converted to HSV color space, and pixels that fall within a predefined blue range are extracted to form a binary mask. A region of interest near the bottom of the frame is selected so the algorithm focuses on the floor area directly in front of the robot. This prevents the robot from seeing blue objects or reflections on surrounding walls and focus only on the blue tape. This mask is then warped into a BEV image using a perspective transform, which makes the geometry of the line more consistent and easier to analyze regardless of distance from the camera.

Within the BEV image, the algorithm computes the centroid of all detected line pixels and, when enough pixels are present, fits a straight line using OpenCV's `fitLine` method [2]. The centroid provides a measure of the robot's deviation from the center of the image, while the fitted line provides an estimate of the tape line orientation relative to the robot's forward direction. These two quantities form the inputs to proportional–derivative (PD) controllers. Their outputs are blended using a tunable weighting factor to compute the robot's angular velocity, while linear velocity is kept constant. This results in smooth forward motion with steering adjustments that respond directly to the calculated centroid and line of best fit of the blue tape pixels.

If too few line pixels are detected to reliably estimate the centroid or orientation, the system enters a controlled "search mode." In this mode, the robot continues moving forward slowly while rotating in the direction of its most recent steering command for a limited time. The most recent steering command is either "right", "left", or "straight" and is defined by the most recent angular velocities published before the tape was lost. This improves the chances of re-acquiring a lost line. If the line is not detected again within a 3-second window, the robot stops to avoid drifting unpredictably. For debugging and evaluation, the system periodically saves images of the raw frame, BEV transform, binary mask, and line-fit visualization.

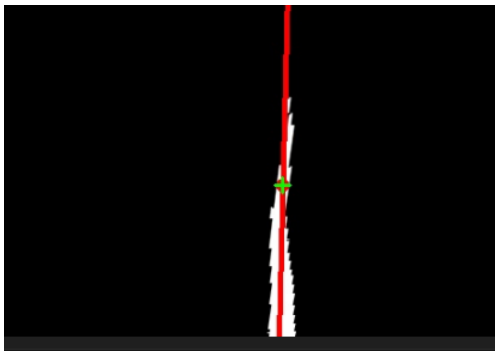


Fig. 4. Line following algorithm applies a binary mask to blue tape

B. SLAM Map Generation

The SLAM subsystem in this project is implemented using the `slam_toolbox` [3] library, which enables real-time 2D SLAM to build an occupancy grid that the robot can navigate through. In our pipeline, the LiDAR sensor publishes `/scan` topic data which is processed by `slam_toolbox` [3]. `slam_toolbox` [3] performs laser scan matching with a Ceres [8] Solver, a method of non-linear optimization, to align scans. Using this scan matching, the `map`, `odom`, and `base_link` frames are updated to maintain a stable and consistent transform structure, allowing the robot to locate itself in the continuously updating map as shown in the figure below.



Fig. 5. `slam_toolbox [3]` mapping lap.

Operating in slam_toolbox [3] *mapping* mode allows the system to incrementally build the 2d occupancy grid map as described above and also performs loop closure when laser scans indicate the robot is visiting a previously explored area. In this way, previously visited areas can be recognized and corrected through global optimization. Loop closure ensures that drift is minimized over time and that the robot's long-term pose estimate remains globally consistent as it moves throughout the environment.

After loop closures stabilizes over one lap of the course, we export:

- map.pgm (occupancy grid data)
- map.yaml (parameter file for map)
- pose_graph.data (serialized graph)

The resulting maps, an example of which is shown in Figs. 5 and 6 below, are used for path planning.

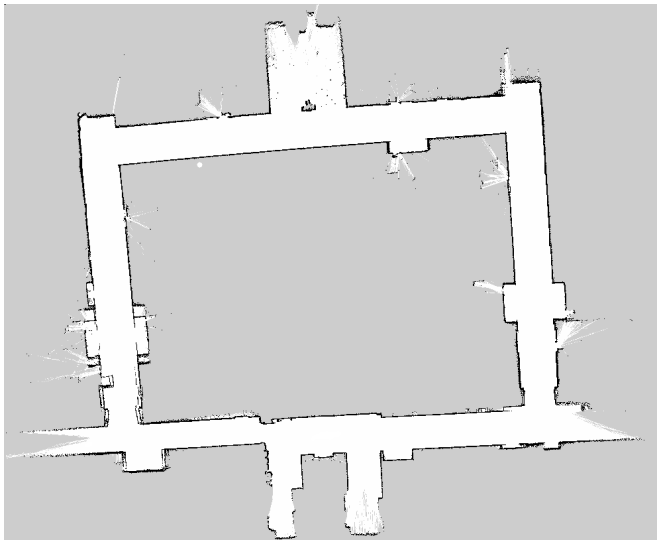


Fig. 6. Map generated using slam_toolbox [3] from our robots line following / mapping lap.

C. Path Planning and Navigation using NAV2

After completing mapping with slam-toolbox [3], the resulting 2D occupancy grid (a grayscale map showing free space, obstacles, and unknown areas) is saved and used as the fixed global map for all navigation tasks. This is passed into the NAV2 [4] navigation framework that handles localization and global path planning.. The *map.pgm* file is viewed in Rviz2 [19] - ROS2's visualization tool [5] that allows users to view maps, sensor data, and robot poses in real time, and to interactively place navigation goals within the environment. When initializing our NAV2 [4] path planning algorithm, users manually place an initial 2d pose, followed by a sequence of goal poses arranged around the indoor track. These goal poses define the path that the robot needs to follow to complete a full loop. NAV2's [4] navigation pipeline, which is activated whenever a goal pose is issued, uses the stored map to compute routes between each point. Localization during navigation was performed entirely by NAV2's [4] Adaptive Monte Carlo Localization (AMCL) [7] module. AMCL [7] is a probabilistic algorithm that maintains many possible pose hypotheses, called particles, and compares predicted sensor readings to the real laser scan data. The algorithm assigns higher importance to particles that match the environment well. At startup, the user supplies an approximate pose through Rviz2, [19] and AMCL [7] refines this estimate continuously as the robot moves. Because AMCL [7] operates on the static occupancy grid rather than performing live mapping, it remains stable and drift-free throughout the indoor loop. The robot uses the default parameters found in the NAV2 [4] launch file for localization purposes. After localization occurs, NAV2's [5] global planner generates a path across the saved occupancy grid, ensuring that the route avoids walls and other obstacles. NAV2 publishes this path to the /path topic and a custom path follower node extracts poses from that path, executing velocity commands to navigate towards each pose in order. This path follower node is tuned to utilize a lookahead distance, following poses further in the distance.

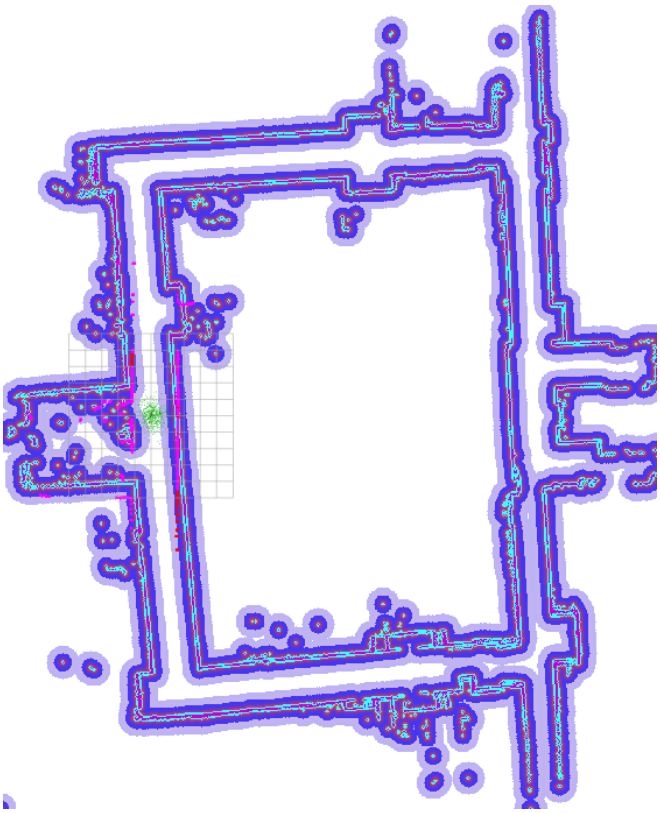


Fig. 7. Map generated using slam_toolbox [3] showing the particle filter locating the robot, the global cost map in purple, and the lidar laser scan in pink.

IV. Camera Navigation (Optional Challenge)

Our team was able to succesfully complete the camera navigation challenge. However, the competition outlines that OpenCV [2] should not be used for camera based line following. Our team did implement two OpenCV functions- one for binary mask computation and one for warping of the image. All other functions in the line following algorithm

were of our own design. As described in the implementation for the line following algorithm, our robot first obtains a raw image from the Realsense camera then applies an OpenCV HSV pixel mask to distinguish blue tape from other objects. Our algorithm calculates a pixel centroid and best fit line for navigation and finally we control steering correction signal with a PD controller to move along the blue tape line.

A threshold calibration procedure was performed in controlled lighting conditions. However, depending on the time of day new masks were needed to correct for lighting conditions.

V. Obstacle Avoidance

After a map is built, new obstacles are detected from LiDAR scans. The Nav2 [4] package reroutes around obstacles by constantly updating the best possible path for our robot to take within the parameters we set for our global costmap function.

VI. Telemetry Visualization Interface

We did not attempt to create a program to act as a cohesive interface to display relevant information as the robot runs. However, instead, we echo information through ROS2 when required with custom logging. Additionally, we are able to utilize RViz2 [19] to visualize slam-toolbox [3] as well as NAV2 [4] working. For our line following algorithm, we also are able to view the raw and masked images in a seperate file we created ourselves.

VII. Experimental Evaluation

To determine our controller performance, we tested several metrics to ensure that our autonomous vehicle was able to meet basic performance thresholds.

A. Mapping Quality

We evaluated:

- Occupancy grid accuracy
- Loop closure reliability
- Localization drift over time

The occupancy grid accuracy worked well, but did experience difficulties with changing obstacles- typically being people walking down the hallway. In cases where low-quality data was received, we simply generated a new map using the line follower algorithm with `slam-toolbox` [3] The algorithm also typically correctly found correct loop closure points but it was necessary to find enough feature points to reliably close the map loop. In addition, the localization worked well but had issues on the corners of the lap due to rapid changes in the Lidar scans and Nav2 needing to find matching features. With increased Lidar mapping rate or reducing the rover speed these issues could be mitigated to some extent.

Over the extent of our testing we found the following lap times to be our best:

Trial	Time (s)	Collisions	Notes
1	195	0	Mapping Lap
2	160	0	Fastest lap

TABLE I
SUMMARY OF TIME TRIAL PERFORMANCE.

VIII. Discussion

A. Discussion

Our autonomous robot ultimately demonstrated the ability to navigate the indoor track using vision-based tape following, generate a quality map of the course, and navigate through multiple laps of the indoor track using the aforementioned map. The blue tape follower algorithm developed by our team was effective in navigating the course when lighting conditions were stable. The algorithm correctly detected blue tape consistently and produced smooth, reliable motion even through sharp corners and gaps in the tape line. For map-based navigation, we were able to efficiently implement `slam_toolbox` [3] to create a 2D-occupancy grid of the track and save this map for later use. After providing an initial pose in RViz2 [19], the `AMCL` [7] module localized the robot on the saved map, and `NAV2` [4] generated global paths through RViz-defined [19] waypoints. This pipeline enabled the robot to complete full loops around the track using only the stored map, LiDAR data, and our `NAV2` [4] integration using waypoints.

While the competition goals were achieved, several limitations of our controller affected the consistency and performance of our autonomous vehicle. The tape follower occasionally struggled when unintended blue objects or reflections appeared in the camera view, causing the robot to drift toward false detections. The color mask was also sensitive to lighting changes, which sometimes caused parts of the real tape to be missed. SLAM mapping was very successful, but at times poor pose estimates or incorrect loop closures produced distorted maps. Our team believes that these inconsistencies in SLAM were often due to changing environmental obstacles, particularly people walking around the LiDaR sensor. Even with a good map, the localization of the robot sometimes failed, which would cause the robot to veer off course and collide into walls or other obstacles. The Lidar-based localization was sensitive to realtively small environmental changes. For example, we would have problems at times if a door was opened that was not opened in our mapping phase, or if large numbers of people were walking around the robot. Because of the difficulties in localization in the map environment, it was necessary to lower our speed so that the robot would have the opportunity to readjust if its localization became momentarily lost or uncertain. Doing so improved the consistency of the robot's ability to complete laps smoothly but at the cost of speed.

Probably the greatest challenge of the competition was integrating multiple ROS2 packages [5] along with our custom algorithms. This required careful configuration of TF

frames, topics, parameters, and nodes. Our team spent much of our time trying to create a pipeline for all these packages to work in conjunction. In addition to the difficulties in the software and pipeline challenges, several hardware issues affected the consistency of the autonomous vehicle performance. Most notably, inconsistent battery voltage sometimes affected motor output and navigation reliability. In other cases, batteries would die quickly and need to be replaced.

Future work will focus on addressing these limitations. For the tape follower, we plan to improve robustness to lighting changes and reduce false detections by refining the color filtering and exploring alternative features beyond color alone. For map-based navigation, we aim to improve the consistency of command velocities so the robot can reach goal poses more accurately and without any human intervention. Additional tuning of `NAV2` [4] controllers, or even custom controller development, may help achieve smoother, faster trajectories. We are also interested in creating a system that can switch automatically between tape following and SLAM-based navigation, allowing the robot to seamlessly transition betwen "mapping" and "navigation" modes without any additional work being done by the user.

IX. Conclusion

This project successfully integrated SLAM mapping, vision-based navigation, and global path planning into a functional autonomous system for indoor course navigation. Our implementation demonstrated reliable map generation using SLAM Toolbox [3], robust reactive control through the Line Following algorithm, and effective Nav2 [4] integration for waypoint-based planning. The dual-mode architecture enabled flexible operation across different competition scenarios. While the system achieved autonomous lap completion, reliable localization during navigation remains a challenge requiring future refinement. Addressing AMCL [7] parameter tuning and pose initialization will improve navigation consistency and reduce dependence on manual intervention. This work establishes a foundation for fully autonomous, competition-ready mobile robotics.

References

[1] Amazon Web Services, "AWS DeepRacer," 2024. [Online]. Available: https://aws.amazon.com/deepracer

[2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[3] S. Macenski, "SLAM Toolbox: SLAM for the dynamic world," 2020. [Online]. Available: https://github.com/SteveMacenski/slam_toolbox

[4] S. Macenski, F. Martín, R. White, and J. Clavero, "The Nav2 Navigation Stack," 2020. [Online]. Available: https://navigation.ros.org

[5] Open Source Robotics Foundation, "ROS 2: Robot Operating System," 2024. [Online]. Available: https://docs.ros.org

[6] Intel Corporation, "Intel RealSense Depth Camera D435i," 2024. [Online]. Available: https://www.intelrealsense.com

[7] D. Fox, "KLD-Sampling: Adaptive Particle Filters," in *Proc. Advances in Neural Information Processing Systems*, 2001.

[8] S. Agarwal, K. Mierle, and Others, "Ceres Solver," Google, 2023. [Online]. Available: http://ceres-solver.org

[9] Slamtec, "RP-LIDAR A1 360° Laser Scanner," 2024. [Online]. Available: https://www.slamtec.com

[10] Holybro, "Pixhawk 4 Mini Flight Controller," 2024. [Online]. Available: https://docs.px4.io

[11] L. Meier et al., "MAVLink: Micro Air Vehicle Communication Protocol," 2024. [Online]. Available: https://mavlink.io

[12] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997.

[13] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

[14] T. Foote, "tf: The Transform Library," in *IEEE International Conference on Technologies for Practical Robot Applications*, 2013.

[15] P. Geneva et al., "OpenVINS: A Research Platform for Visual-Inertial Estimation," *ICRA*, 2020.

[16] G. van Rossum and F. L. Drake, *Python 3 Reference Manual*, Python Software Foundation, 2009.

[17] Canonical Ltd., "Ubuntu 22.04 LTS Documentation," 2024. [Online]. Available: https://ubuntu.com

[18] Raspberry Pi Foundation, "Raspberry Pi 4 Model B," 2019. [Online]. Available: `https://www.raspberrypi.com/products/raspberry-pi-4-model-b/`

[19] Open Robotics, "RViz2: 3D Visualization Tool for ROS 2," 2020. [Online]. Available: `https://github.com/ros2/rviz`

Appendix

- SLAM Package Implementation (8 points)
- Camera Navigation (3 points)
- Obstacle Avoidance (2 points)
- Telemetry Visualization Interface (1 point)

Code and documentation:
`https://github.com/jnealo32/tne-robotics`